

哈夫曼树加密与解密实验报告

2026 年 4 月 18 日

1 实现原理（核心思想）

本程序基于**哈夫曼编码（Huffman Coding）**原理实现字符的加密与解密。哈夫曼编码是一种基于贪心策略的变长前缀编码方法。

1. **核心原则**：出现频率（权重）越高的字符，其编码长度越短；频率越低的字符，编码长度越长。以此实现数据的压缩与加密。
2. **前缀码特性**：任何一个字符的编码都不是另一个字符编码的前缀，保证了解密（解码）时的唯一性，不会产生歧义。
3. **树形结构**：通过构建一棵最优二叉树（哈夫曼树），规定左分支代表 "0"，右分支代表 "1"，从根节点到叶子节点的路径即为该叶子节点所代表字符的编码。

2 数据结构设计

程序采用**数组模拟指针（静态链表）**的方式来存储哈夫曼树。定义了结构体 `HTNode`，包含以下属性：

- `ch`：存储对应的字符（仅叶子节点有效）。
- `weight`：字符出现的频率（权重）。
- `left / right`：左、右孩子节点在数组中的索引（默认为 `-1`，表示无孩子）。
- `parent`：父节点在数组中的索引（默认为 `-1`，表示无父节点）。

对于 n 个叶子节点（本程序中 $n = 27$ ，即 26 个小写字母加 1 个空格），哈夫曼树总共有 $2n - 1$ 个节点。

3 核心代码思路（模块解析）

3.1 寻找最小节点模块 (Select 函数)

- **思路：**遍历当前哈夫曼树数组中所有尚未分配父节点 (`parent == -1`) 的节点。
- **操作：**通过一次遍历，维护并找出权重 (`weight`) 最小和次小的两个节点的索引 `s1` 和 `s2`，供建树时合并使用。

3.2 构建哈夫曼树 (CreatHT 函数)

- **思路：**自底向上构建二叉树。
- **操作：**从索引 n 开始到 $2n - 2$ 结束，循环 $n - 1$ 次。每次调用 `Select` 取出当前权值最小的两个孤立节点，将它们作为左右孩子合并成一个新的父节点。新父节点的权值为两孩子权值之和，并更新孩子节点的 `parent` 指针。

3.3 生成哈夫曼编码 (HFcoding 函数)

- **思路：**自底向上（从叶子节点到根节点）逆向求编码。
- **操作：**遍历所有前 n 个叶子节点，顺着 `parent` 指针一直向上回溯直到根节点。在回溯过程中，如果当前节点是其父节点的左孩子，则在编码字符串前追加 "0"；若是右孩子，则追加 "1"。最终得到每个字符的二进制编码串。

3.4 文本加密/编码 (Encode 函数)

- **思路：**字符替换。
- **操作：**遍历用户输入的字符串，在字母表数组中查找对应的字符索引，然后将对应的哈夫曼编码追加到结果字符串 `code` 中，实现明文到密文的转换。

3.5 文本解密/解码 (Decode 函数)

- **思路：**自顶向下遍历哈夫曼树。
- **操作：**从哈夫曼树的根节点（索引为 $m - 1$ ）开始，逐位读取密文字符串（"0" 或 "1"）。
 - 遇到 "0" 则走向左孩子，遇到 "1" 则走向右孩子。
 - 当走到叶子节点（即 `left` 和 `right` 均为 `-1` 时），说明成功解析出一个字符，将其拼接到明文字符串中。
 - 随后将指针重新拨回根节点，继续解析密文的下一位，直到密文读取完毕。

4 主程序执行流程 (main 函数)

1. **初始化**: 定义 27 个字符的字母表及其对应的频率分布权重, 动态分配节点数组和编码数组的内存。
2. **预处理**: 调用建树和编码函数, 生成完整的哈夫曼树及编码表。
3. **输入处理**: 接收用户输入的明文字符串, 统一转换为小写字母以防匹配失败。
4. **加密与解密**: 依次调用 Encode 获得由 "0" 和 "1" 组成的加密串, 再调用 Decode 将加密串还原为明文, 并分别输出打印结果。
5. **资源释放**: 释放动态申请的内存, 防止内存泄漏。

5 实验结果展示

以下为本程序的运行结果截图, 展示了哈夫曼编码表的生成以及对输入字符串进行加密与解密的过程。

```
e:001
f:101001
g:101010
h:11011
i:0101
j:1011110
k:1011111
l:10011
m:101011
n:0110
o:1000
p:101100
q:1101000
r:0000
s:0001
t:1100
u:01001
v:1101001
w:101101
x:1101010
y:101110
z:1101011
:111
Enter your words to encode:hello sjtu
codes are:110110011001110011100011100011011110110001001
Words are:hello sjtu
```

图 1: 实验结果截图一

```
e:001
f:101001
g:101010
h:11011
i:0101
j:1011110
k:1011111
l:10011
m:101011
n:0110
o:1000
p:101100
q:1101000
r:0000
s:0001
t:1100
u:01001
v:1101001
w:101101
x:1101010
y:101110
z:1101011
:111
Enter your words to encode:I am saier
codes are:010111101111010111110001011101010010000
Words are:i am saier
```

图 2: 实验结果截图二